NMAM INSTITUTE OF TECHNOLOGY, NITTE
Department of Information Science and Engineering

OPERATING SYSTEMS

PROGRAMMING ASSIGNMENT REPORT

AI-Assisted Synchronization Algorithms with Program-Generated HTML Simulation

Readers-Writers Problem and Dining Philosophers Problem
(Semaphore-based and Monitor-based implementations in Python)

SUBMITTED BY

Name	LAHASYA L KANCHAN
Registration / USN	NNM24IS114
Course	Operating Systems
GitHub Repository	https://github.com/lahasya-lk/OS-Synchronization-Assignment

SUBMITTED TO

Dr Jason Elroy Martis
Associate Professor
Dept Information Science and Engineering,Nitte

1. Problem Statement and Objective

1.1 Problem Statement

In a multithreading OS, there might be several threads running at the same time and accessing the shared resource. In case of such threads, their operations may conflict with each other and result in incorrect outcomes. There may be race conditions, violation of mutual exclusion, deadlocks or starvation caused by unsynchronized execution of threads. That is why synchronization should be used to make sure that threads' interactions with shared resources are properly regulated.

The project considers two traditional synchronization problems for processes: the Readers-Writers problem and the Dining Philosophers problem. In the Readers-Writers problem, there are several readers who have access to a shared resource at the same time, but the writer needs exclusive access. The aim is to achieve maximal concurrency between readers, but not to let the writer have access to the resource. Both semaphores and monitors are used to implement the Readers-Writers problem.

The second problem is the Dining Philosophers problem where four philosophers eat from the plate using four chopsticks around the table. Each philosopher needs two adjacent chopsticks to eat.

The problem arises from the situation where all philosophers try to get access to the resources in an improper sequence leading to deadlock where no philosopher is able to move further. Hence, the project incorporates deadlock prevention schemes and considers the probability of starvation. The semaphore solution involves room semaphore and individual chopstick semaphores whereas the monitor solution uses atomic test and set mechanism with randomized back off scheme.

Apart from implementing synchronization algorithms, the objective of the project is to make their behavior more comprehensible by way of simulating it in the form of an HTML page generated by the programs. Instead of making any kind of pre-recorded animation, the programs record synchronization activities such as entering a critical section, getting resource access, waiting, eating, writing, and releasing resources. After recording the activities, the recorded information is presented in the form of an HTML page which can simulate execution with the help of control like start, pause, step, reset, speed selection, and timeline.

Hence, the core issue that the project tries to solve is the implementation and comparison of synchronization techniques based on semaphore and monitor methods.

1.2. Objectives

Objectives of the project include:

1. Implementing the Readers-Writers synchronization problem with the use of semaphores and monitors, whereby several readers may use the resource but only one writer can use it at a time.
2. Implementing the Dining Philosophers problem with 4 philosophers using both semaphore-based and monitor-based synchronization methods.
3. Implementing mutual exclusion and avoiding race conditions when there is concurrent access to shared resources by several threads.
4. Avoiding deadlocks in the Dining Philosophers problem through the use of appropriate synchronization techniques, such as the room semaphore and test-and-set operation.
5. Detecting starvation and fairness based on how various synchronization policies prefer readers, writers, or philosophers.

6. Logging the actual execution of the program through recording synchronization events and state changes while executing.
7. Generating an HTML visualization from the logged execution in order to see the synchronization process step by step through a web browser.
8. Comparing semaphores and monitors with regard to implementation, resource management, deadlock avoidance, starvation, and clarity of design.
9. Testing implementations for various numbers of threads and delay values and checking for correctness using invariant checking and execution trace.
10. Gaining an understanding of process synchronization through linking theoretical operating system.

2. Readers-Writers: Semaphore Solution

Implementation one solves the Readers-writer problem through the use of semaphores. In the readers-writers problem, there exists a shared resource which can be accessed simultaneously by many readers at a certain period of time but a writer needs to have exclusive access to the shared resource. The shared resource in the project is defined by an integer value. Multiple readers can access the value, however, the writer should operate alone. Therefore, the major concern here is how to give as much concurrency as possible for the readers but not letting any writer into the critical section.

There exist two binary semaphores used in this solution: mutex and wrt. The mutex semaphore is used to protect the read_count variable which gives the number of readers accessing the shared resource. The wrt semaphore is used to protect the shared resource itself and to ensure exclusive access for the writer to the resource.First reader acquires mutex, increments read_count, then wrt. As a result, writers will not access the resource until all readers will finish the operations with it. After acquiring mutex, the first reader will increment read_count and release mutex. It performs the reading operation and other readers.

When a reader finishes, it again acquires mutex and decreases read_count. If it is the last reader, it releases wrt, allowing a waiting writer to access the resource. A writer directly waits for wrt, performs its update while holding the semaphore, and releases it after completing the operation.This implementation follows a readers' preference approach. Since new readers are allowed to enter while other readers are already active, a continuous stream of readers can keep a writer waiting for a long period[1].This means that writer starvation is possible in theory. However, the design provides efficient concurrent access for readers.

The implementation also deliberately creates a small delay between the writer reading and updating the value. This makes synchronization problems easier to observe during testing. If the writer's exclusive access were removed, two writers could read the same old value and overwrite each other's updates. Therefore, the wrt semaphore is important for maintaining correctness.When the reader completes, it once again gets the mutex and reduces the value of read_count. In case this reader is the last one, it releases the wrt, thus letting any waiting writer access the resource. The writer waits for the wrt directly, does the update while having the semaphore and then releases it.

In this case, the writers' preference strategy is implemented. As new readers can come into play while there are still active readers, a stream of readers can make the writer wait for a long time. Hence, writer starvation can theoretically occur[2][3].Yet, there is efficient concurrency for readers provided by this design.Also, there is an intentional delay between the reading and updating of the value done by the writer. It helps to see synchronization issues while testing the code. In case the exclusive access of the writer was not provided, two writers would be able to use the old value and overwrite the updates of one another. Hence, the wrt semaphore is essential here.The code captures the events related to the readers' entrance into and exit from the critical section, as well as acquiring and releasing of semaphores and writers' updating of the value.

These events are later used by the HTML simulation to show how multiple readers can operate together while a writer has to wait.

3. Readers-Writers: Monitor Solution

The second example illustrates a solution for the Readers-Writers problem by employing the monitor approach. As opposed to the first one, all of the synchronization takes place within a single object similar to the monitor. In order to simulate such an approach in Python, which does not have the concept of monitor, both the lock and the condition variable have been used.

This implementation uses three variables active_readers, writing and waiting_writers. The active_readers variable contains the number of readers currently using the resource. The writing variable is set to True if there is any writer inside the critical section of code. Waiting_writers variable contains writers, which are waiting to get into the critical section.

Two condition variables can_read and can_write are used to synchronize the reader and the writer processes. When the reader is about to use the resource, it calls start_read() method. If there is a writer currently writing or there is a writer waiting, the reader waits until the signal from can_read comes. Otherwise, it increments active_readers variable and continues working.

After the reader has finished its work, it calls end_read() method, which decrements active_readers variable. If the current reader is the last active reader, then the process.

However, a writer goes through a different order. If it needs to write, it increments the waiting_writers and waits until there are no active readers and no other writer is currently writing. When the writer is ready to proceed, it sets itself to be active and writing is set to true. Once it has finished its operation, the writer clears the writing flag and wakes up one of the waiting writers or readers.

The implementation includes the notion of writer preference. Once the writer starts waiting, no new readers can enter. This differs from the semaphore version of readers-writer, where reader preference is included. The reason for this is that we want to decrease the chances of writers' starvation due to an endless flow of readers.

Conditions are checked inside while-loops and not using if-statements because when a thread wakes up, it cannot know whether the condition holds because another thread may have modified the shared data in the meantime.

This approach makes the synchronization strategy clearer due to the fact that all the shared data, lock and condition variables are grouped together. On the other hand, it imposes its own set of responsibilities like applying the proper conditions for waiting and notifying the right threads.
The HTML simulation keeps track of the changes in the state and shows the behaviour of preference to writers. This can be seen when the reader asks for permission to enter even when there is a writer who is already waiting, he/she will not be allowed until the waiting writer finishes.

4. Dining Philosophers (4): Semaphore Solution

In the third implementation of the solution, semaphores are used to solve the Dining Philosophers problem. There are four philosophers arranged around a table that contains four chopsticks, one chopstick between each pair of adjacent philosophers. A philosopher cycles between thinking, becoming hungry, picking up chopsticks, eating, and dropping the chopsticks.

Each philosopher requires two chopsticks in order to be able to eat: the left-hand chopstick and the right-hand chopstick. The main problem with this is that if all four philosophers pick up one chopstick and wait for another, they will all be blocked. This leads to a circular wait and thus deadlock.

To avoid such deadlock situations, four binary semaphores are used for each chopstick. The value of each semaphore equals one because it means that the chopstick is available. Furthermore, there is a counting semaphore called room with value 3.

Before trying to pick up chopsticks, a philosopher needs to enter the room using the room semaphore. Because the room allows at most three philosophers to compete for chopsticks simultaneously, there can never be all four philosophers holding one chopstick. This breaks the circular-wait requirement for the classical deadlock case.

The philosopher first gets the left chopstick, followed by getting the right chopstick. On having gotten the two chopsticks, the philosopher gets into the eating state. Upon finishing eating, the philosopher lets go of the right chopstick, followed by letting go of the left chopstick and finally letting go of the room semaphore. This allows another waiting philosopher an opportunity to try to acquire the resources.

A useful characteristic of this approach is that the simulation will enable the philosopher to be holding one chopstick while waiting for the other chopstick. This makes it easy to understand the resource allocation procedure visually. This also helps to explain the danger of acquiring one resource at a time without any other safeguard against circular waiting.

The room semaphore is thus the main way through which deadlocks are avoided. In a scenario where all four philosophers were allowed to compete without this limitation, circular wait scenario would arise. However, setting the value of room to three will make it possible for at least one philosopher to acquire the necessary resources and eat.

While the approach solves the problem of deadlock, starvation is not strictly prevented. The semaphores in the Python language cannot strictly enforce first-come-first-served scheduling. This means that a particular philosopher can wait longer than others due to other philosophers obtaining the needed forks before him or her.

In addition, there are certain events that are recorded in the code in order to generate the HTML simulation: being hungry, going into the dining room, taking the forks, eating, and releasing the resources.

5. Dining Philosophers (4): Monitor Solution

Implementation four uses a monitor-based solution with the use of the test-and-set and randomized back-off. In this implementation, the attempt to gain two required chopsticks by the philosopher is performed as a whole.

There is an array owner[0..3] which keeps information on which philosopher owns each chopstick[4].There is a monitor lock for this shared data and there is a private condition variable for each philosopher.

The core function is test_and_set(i). Whenever philosopher i wants to eat, it checks whether both of his chopsticks are free and if they are, claims them both. The most interesting thing here is that this check and assign is performed under the monitor lock. This means that another philosopher cannot perform any action in between this check and assign. Hence, philosopher i does not gain one chopstick and wait while owning it. Two chopsticks are obtained together or not obtained at all. Thus, hold-and-wait condition is eliminated which removes circular deadlock possibility.

In the case where the test-and-set operation is unsuccessful because one or both chopsticks are currently in use, the philosopher does not continuously keep on checking for the availability of the resources in a busy loop. It incorporates a back-off approach. The philosopher will wait for a delay that is randomized, and the waiting increases with the number of retries according to the following approximation:

base × 2^attempt × random(0.5, 1.5)

The usage of randomized back-off minimizes collisions between philosophers since in the case where all the philosophers retrying were to attempt to get access to the resource at the same time, they would repeatedly compete for the same chopsticks. By making the process randomized, the collision can be reduced. The condition-variable wait releases the monitor lock while the philosopher waits. Hence other philosophers will be able to continue executing and acquiring resources. When the neighboring philosophers release their chopsticks, they can be notified to retry earlier without necessarily waiting for the whole delay[5].This provides an alternative solution to the Dining Philosophers problem compared to the semaphore solution. The semaphore solution prevents deadlock by controlling the number of philosophers that can compete for resources.In contrast to the monitor method, where the problem of partial allocation is resolved because it takes possession of both chopsticks atomically.

Simulation demonstrates this distinction clearly. A philosopher unable to take possession of both chopsticks does not show one chopstick and wait for another. Rather, there is shown a failure of the test-and-set procedure and a waiting period due to back-off.Randomized back-off decreases the chances of repeating starvation, but it does not give any kind of mathematical guarantee of the fairness of access to resources. A philosopher can simply get unlucky with this approach and be starved again and again for its chopsticks.

6. Event Recording and HTML Generation

The demonstration of synchronization does not involve a dedicated, manually created animation, but each Python script tracks the key events that happen during the actual running of the synchronization process and creates a visualization based on these events. The visualization will be therefore directly related to the behavior of the synchronization algorithm.

While running, all scripts keep the event log, which records an event together with relevant details whenever an important state change happens – who performed the action, what was done, what state the object was in and when it happened. The examples of such events include: starting/finishing the work of a reader, waiting/writing for a writer, a philosopher becoming hungry, taking a chopstick, starting to eat, freeing resources, or going into waiting state. Such event logging is built into the synchronization logic, which means that the visualization corresponds to the actual run of the program and not some predetermined order of events. For instance, while the Readers-Writers problem with semaphores allows two readers to read at once, it will be logged twice.In a similar fashion, if the writer has to wait due to readers being active, this waiting activity is recorded and can be viewed later in the simulation.The recorded activities are then converted into an HTML file. The generated HTML contains the information that would allow the execution to be displayed in the browser. The JavaScript language will be used to process the recorded activities and update the display as the simulation proceeds. The visualization allows displaying the current state of the threads/philosophers and the resources they use.

In the case of the Readers-Writers problem, the HTML simulation could display activities of readers accessing the common resource, activities of writers waiting for access to it, and activities of writers making updates. This will allow observing the differences between implementations of the problem with semaphores and monitors. The preference of readers would be noticeable in the semaphore implementation when several readers could proceed while a writer was waiting[6][7]. The writer preference will be demonstrated in the monitor implementation of the problem.

In the case of Dining Philosophers problem, visualization presents the states of philosophers and ownership of chopsticks. Philosopher state changes involve thinking, becoming hungry, waiting for resources, eating, and releasing the chopsticks. The semaphore implementation reveals how room semaphore prevents more than four philosophers from fighting for chopsticks. In monitor implementation test-and-set behaviour and back-off period are illustrated when both needed chopsticks cannot be taken.The resulting HTML also offers the ability to investigate the execution in detail. The execution can be run, paused, reset, or stepped frame-by-frame in order to observe individual synchronization actions. The ability to control the speed of the execution and the timeline can help to investigate the execution process.

Overall, event logging ties together the program execution, synchronization mechanism, and visualization. Rather than animating what should take place, the HTML file visualizes what was going on in the course of program execution. It helps to comprehend and compare the concepts of mutual exclusion, waiting, resource allocation, deadlock prevention, and starvation.

7. Execution Traces

7.1 Readers-Writers Using Semaphore

More than one reader can enter the critical section at the same time as long as the first reader has obtained the writer lock. The writer W0 must wait until all readers have exited and released the lock so that writing will not overlap with reading.

Example:

0.00s	P3	thinking (meal 1/3)
0.15s	P2	is hungry: wait(room)
0.15s	P2	entered room: wait(left chopstick)
0.15s	P2	picked chopstick 2
0.22s	P3	is hungry: wait(room)
0.22s	P3	entered room: wait(left chopstick)
0.22s	P3	picked chopstick 3
0.29s	P3	picked chopstick 0
0.29s	P3	EATING (critical section)
0.30s	P1	is hungry: wait(room)
0.30s	P1	entered room: wait(left chopstick)
0.30s	P1	picked chopstick 1

7.2 Readers-Writers Using Monitor

The monitor implementation controls readers and writers through synchronization conditions and prioritizes waiting writers over incoming readers. W1 can enter the critical section after the active readers exit, and once W1 has completed its work, then W0 can enter the critical section to do its writing operation.

Example:

0.27s	R1	wants to read
0.32s	W0	wants to write
0.43s	R2	EXITS CS
0.54s	R2	wants to read
0.62s	R0	EXITS CS
0.63s	R3	EXITS CS
0.63s	W1	ENTERS CS (exclusive)
0.87s	W1	wrote value 0 -> 1
0.87s	W1	EXITS CS
0.87s	W0	ENTERS CS (exclusive)

7.3 Dining Philosophers Using Semaphore

A philosopher will wait to enter the room first before he acquires both the left and right chopsticks. In case any of the chopsticks is not available, then the philosopher will block his process until it becomes available, thereby eating together as a group and avoiding deadlocks.

Example:

0.00s	P3	thinking (meal 1/3)
0.15s	P2	is hungry: wait(room)
0.15s	P2	entered room: wait(left chopstick)
0.15s	P2	picked chopstick 2
0.22s	P3	is hungry: wait(room)
0.22s	P3	entered room: wait(left chopstick)
0.22s	P3	picked chopstick 3
0.29s	P3	picked chopstick 0
0.29s	P3	EATING (critical section)
0.30s	P1	is hungry: wait(room)
0.30s	P1	entered room: wait(left chopstick)
0.30s	P1	picked chopstick 1

7.4 Dining Philosophers Using Monitor

The monitor implementation uses the test-and-set technique in determining whether both the chopsticks are available before it permits a philosopher to eat. If the attempt fails, the philosopher will block for a random back-off period and then try again.

Example:

0.18s	P2	is hungry
0.18s	P2	test-and-set OK: took chopsticks 2 and 3
0.18s	P2	EATING (critical section)
0.30s	P1	is hungry
0.30s	P1	test-and-set failed (chopsticks busy: P[2]); back off 69 ms (attempt 1)
0.37s	P1	test-and-set failed (chopsticks busy: P[2]); back off 128 ms (attempt 2)
0.41s	P0	is hungry
0.41s	P0	test-and-set OK: took chopsticks 0 and 1
0.41s	P0	EATING (critical section)
0.45s	P3	is hungry
0.45s	P3	test-and-set failed (chopsticks busy: P[0, 2]); back off 114 ms (attempt 1)
0.49s	P2	released chopsticks 2 and 3

8. Race Conditions, Mutual Exclusion, Deadlock and Starvation

Race Conditions

In the Readers–Writers problem, race conditions can occur due to the fact that multiple readers may try to modify the value of read_count simultaneously without a mutex. It may result in an incorrect number of readers and difficulties during wrt releasing. The same situation is observed for writers who can overwrite each other's updates and obtain an incorrect final value. I tried to test that by removing the wrt from the writer program, and the invariant checker indicated over 40 overlaps in each of three attempts[8].The final value was 3 instead of expected 6 because of lost updates. In the Dining Philosophers problem, synchronization helps to exclude simultaneous claiming of one and the same chopstick by two neighboring philosophers.

Mutual Exclusion

All four programs utilize synchronization methods in order to provide protection for their critical sections and avoid conflicting access to shared variables. Only one writer can access the shared data, while multiple readers can read the data simultaneously. In the Dining Philosophers problem, only one philosopher can hold one and the same chopstick at a time. Invariant checker checks those requirements after every event. In all the completed attempts, zero mutual exclusions have been reported.

Deadlock

In the Readers–Writers implementations, deadlock cannot occur as these implementations prevent the circular wait. The readers can start working before other readers get a chance to do so, and only the last one needs to unlock wrt, but it is still accessible to all readers through read_count. The Dining Philosophers implementations avoid deadlock in different ways[9]. The semaphore version uses the room semaphore to regulate the number of philosophers that compete for chopsticks as otherwise, deadlock is quite possible. The monitor version tries to acquire both chopsticks at once so that a philosopher would not be stuck waiting for one. All runs completed successfully, there were no deadlocks.

Starvation

Starvation may still be possible in all these cases since no one of them assures fairness under any circumstances. In the semaphore Readers-Writers solution, a constant influx of readers may starve writers waiting for their turn[10].The use of a turnstile semaphore may solve this problem as it prevents readers from entering if there is a waiting writer. In the monitor-based solution, writers get preference over readers, so readers may be starved if writers enter the critical section continuously. In the Dining Philosophers case, the semaphore solution relies upon scheduling policies of waiting threads, whereas in the monitor solution, there are random delays used. These methods allow the programs to make some progress, but do not assure that there will be no starvation.

9. Semaphore vs Monitor Comparison

Aspect	Semaphore	Monitor
Basic concept	Uses counters and wait() / signal() operations.	Uses a shared structure with locks and condition-based synchronization.
Mutual exclusion	Achieved using binary or counting semaphores.	Achieved through monitor locks and controlled access.
Implementation	Requires careful handling of semaphore operations.	Encapsulates synchronization with shared data and methods.
Readers–Writers	Uses mutex and wrt to coordinate readers and writers.	Uses synchronization conditions to manage access and waiting threads.
Dining Philosophers	Uses a room semaphore and chopstick semaphores to prevent deadlock.	Uses a lock and atomic chopstick allocation with test-and-set and back-off.
Main advantage	Flexible and useful for controlling access to multiple resources.	Provides a structured approach to synchronization.
Main limitation	Incorrect semaphore operations can cause deadlocks or synchronization errors.	Fairness and starvation prevention require additional logic.

Conclusion

This assignment applied and experimented on synchronization of the Readers-Writers and Dining Philosophers problems using semaphore synchronization and monitor synchronization. The execution trails and HTML demonstrations helped in illustrating how threads are synchronized and how they access resources, wait for access, and leave critical section.

It can be observed from the results that both synchronization methods kept the mutual exclusion principle and prevented race condition in the executed instances. The Readers-Writers problem created a final output of 6 with zero invariant violation in both implementations. Likewise, both Dining Philosophers algorithms let all philosophers eat without deadlock occurrence.

The above analysis illustrates that semaphore synchronization allows flexibility and fine-grained control while monitor synchronization is a structured way to handle resource access. Nonetheless, both do not assure the avoidance of starvation problem without the implementation of fairness technique.

10. Screenshots of Generated HTML Simulations

Each screenshot is a frame taken from the page produced by the corresponding program.

10.1 Readers-Writers Using Semaphore

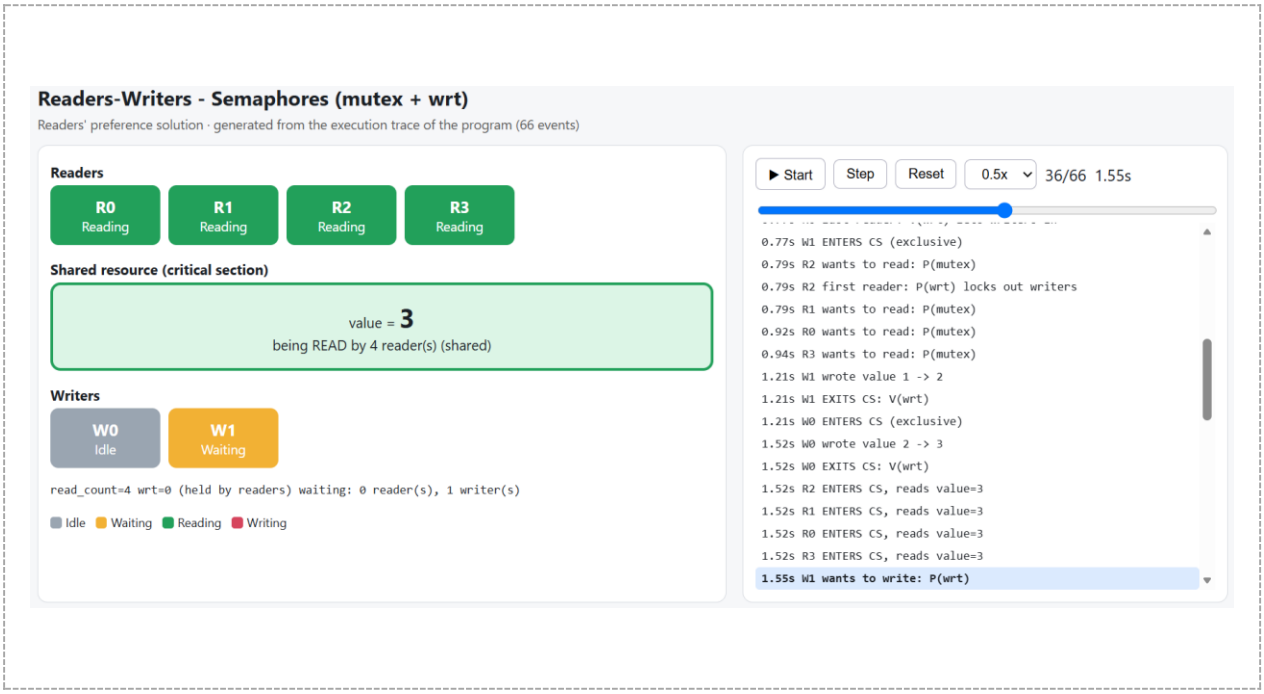


Figure 5: HTML simulation generated by readers_writers_semaphore.py

10.2 Readers-Writers Using Monitor

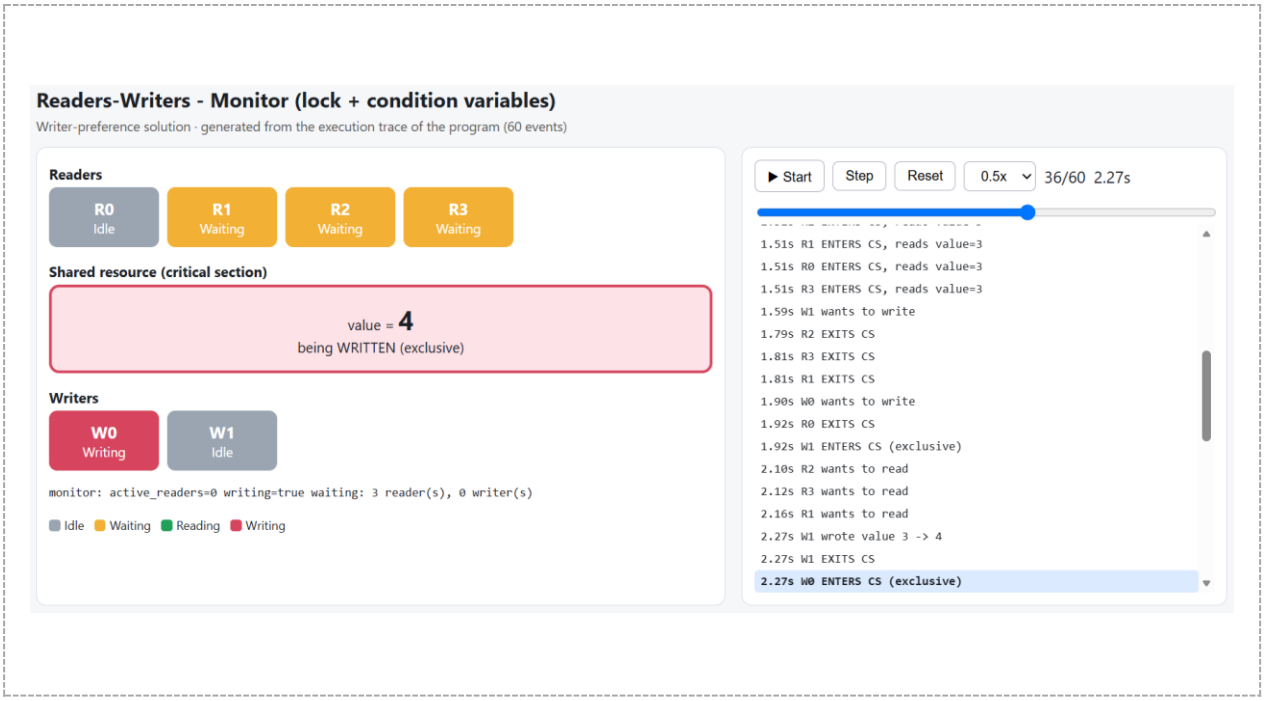


Figure 5: HTML simulation generated by readers_writers_monitor.py

10.3 Dining Philosophers Using Semaphore

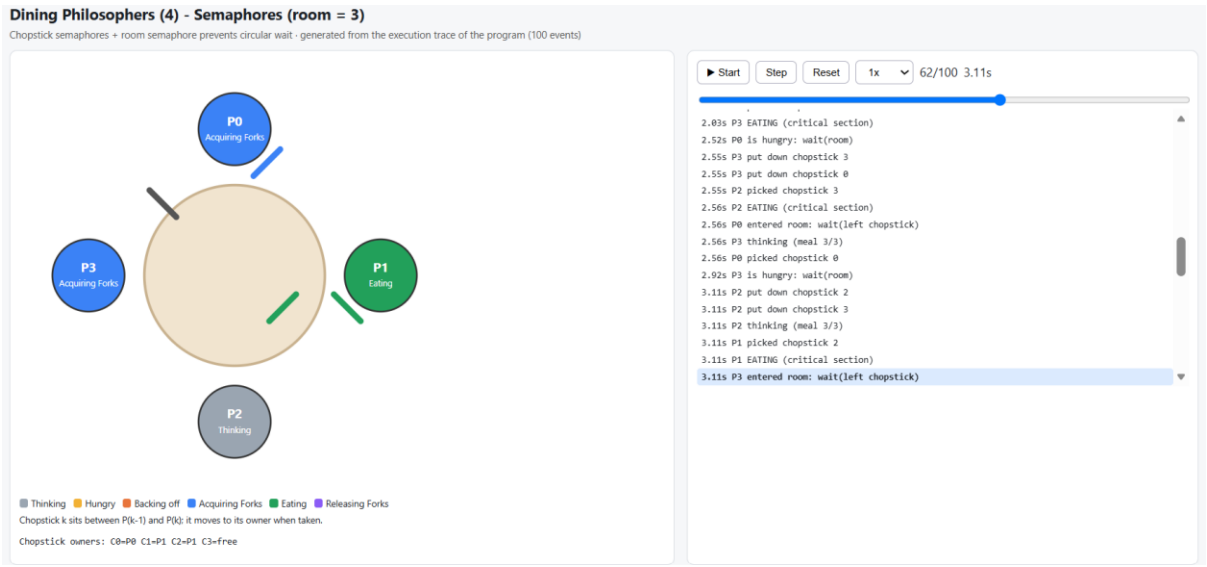


Figure 5: HTML simulation generated by dining_philosophers _semaphore.py

10.4 Dining Philosophers Using Monitor

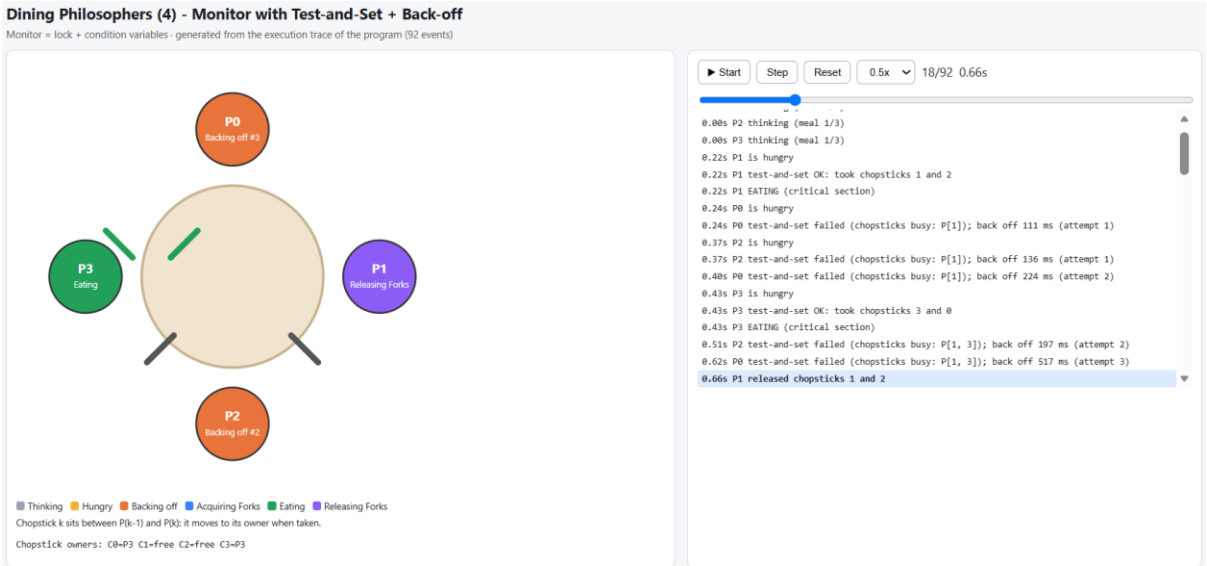


Figure 7: HTML simulation generated by dining_philosophers_monitor.py

11. AI Usage Summary

Assistant Claude (Anthropic) was employed in the process of the development of the assignment in the course of planning, programming, algorithm description, HTML visualisation design and documentation drafting.

The initial prompt requested from the assistant to proceed with the instructions of the assignment and to develop the structure Python programs that work in VS Code and include an explanation. Subsequent prompts addressed questions about file location and order of their execution, where to put the screenshots, how to prevent deadlock in the Dining Philosophers problem (including why three philosophers in the room cannot eat at once) and the report, README, prompt log and reference list. All relevant prompts can be found in ai_prompts/prompts.md.

The testing of the programs consisted in repeated execution with various numbers of threads and delays and verification of the synchronization via violation counters, final values and traces.

Several limitations and problems have been revealed in the course of the development process. The Python semaphores and condition variables do not provide for strict first-in-first-out awakening, hence starvation cannot be fully excluded in the semaphore Dining Philosophers program.

First, the wrong explanation about the room semaphore assumed that three philosophers could eat together, but the right answer is that three may fight, whereas only two can eat. Second, the deadlock detection procedure in the Dining Philosophers programs waits for each thread one after another, which means that a true deadlock might take a couple of minutes before it gets detected. Third, in the assignment text there was some hidden text aimed at AI assistants, telling them to put in their outputs a certain word and not to expose it.

12. References

[1] Python Software Foundation, “threading — Thread-based parallelism,” Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/threading.html

[2] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, Wiley.

[3] Oracle, “Synchronization,” The Java Tutorials. Available: Oracle synchronization concepts

[4] Hoare, C. A. R. Monitors: An Operating System Structuring Concept. *Communications of the ACM*, 1974.

[5] Dijkstra, E. W. Hierarchical Ordering of Sequential Processes. *Acta Informatica*, 1971.

[6] L. Lamport, “The Mutual Exclusion Problem: Part I—A Theory of Interprocess Communication,” Journal of the ACM, vol. 33, no. 2, pp. 313–326, 1986. DOI: 10.1145/5383.5384

[7] Courtois, P. J., Heymans, F. and Parnas, D. L. Concurrent Control with 'Readers' and 'Writers'. *Communications of the ACM*, 1971.

[8] Tanenbaum, A. S. and Bos, H. *Modern Operating Systems*. Pearson. (Classical IPC problems.)

[9] Oracle, “Intrinsic Locks and Synchronization,” The Java Tutorials. Available: Oracle Java synchronization tutorial

[10] K. M. Chandy and J. Misra, “The Drinking Philosophers Problem,” *ACM Transactions on Programming Languages and Systems*, vol. 6, no. 4, pp. 632–646, 1984. DOI: 10.1145/1780.1804

13. Academic Integrity Declaration

I declare that I have personally tested and understood the submitted programs. I have disclosed the AI tools and external resources used in developing this assignment. I can explain the synchronization logic, generated HTML simulation, and modifications made to the code. I have not knowingly submitted another student's work or unacknowledged third-party material as my own.

Name: LAHASYA L KANCHAN

USN: NNM24IS114

Signature: _____

Date: _____